

Actividad 02 — Construcción de Portafolios Óptimos

Enfoque Incremental: EDA $\rightarrow$ Markowitz (Óptimo) $\rightarrow$ Simulación (Validación) $\rightarrow$ Servicing
(Manual vs Desarrollo Asistido con IA)

Sebastián Egaña Santibáñez (segana@fen.uchile.cl)
Nicolás Leiva Díaz (nleivad@fen.uchile.cl)

2026-09-24

Descargas y Enlaces

Enlaces del profesor

<https://segana.netlify.app>

<https://github.com/sebaegana>

<https://www.linkedin.com/in/sebastian-egana-santibanez/>

Introducción: El Problema

Contexto: Tienes acceso a 5 activos financieros: SPY (S&P 500), EFA (mercados desarrollados), IJS (small-cap), EEM (mercados emergentes), AGG (bonos). Deseas construir un portafolio que maximice el retorno relativo al riesgo, pero **no sabes qué pesos asignar** a cada uno.

Pregunta central: - ¿Cuál es la combinación *óptima* de pesos? - ¿Cómo *verificar* que esa solución es realmente óptima? - ¿Cómo *publicar* ese resultado para que otros lo usen?

Tu rol: seguir un pipeline de datos completo: 1. **Parte 1 — EDA:** entender los datos y correlaciones entre activos 2. **Parte 2 — Markowitz:** calcular la frontera eficiente (solución analítica exacta) 3. **Parte 3 — Simulación:** generar miles de portafolios aleatorios para validar que Markowitz es realmente óptimo 4. **Parte 4 — Comparación:** ¿cuán buena es la nube simulada vs la frontera exacta? 5. **Parte 5 — Servicing:** publicar el resultado como tu propio proyecto de Quarto en GitHub Pages

En cada parte, verás dos enfoques: **Manual** (escribes cada línea de código) y **Asistido con Claude Code** (acelera 5-10x).

Parte 1 — EDA: Explorando los Datos y Correlaciones

Enfoque Manual (Tradicional)

Sin IA, siguiendo la estructura clásica:

Paso 1: Cargar librerías y datos

```
library(readr)
library(tidyverse)
library(skimr)
library(corrplot)
library(kableExtra)

portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

head(portafolio_retornos)
str(portafolio_retornos)
```

Tiempo: ~3 minutos.

Paso 2: Estadística descriptiva

```
summary(portafolio_retornos |> select(-fecha))
skim(portafolio_retornos |> select(-fecha))

stats_por_activo <- portafolio_retornos |>
  select(-fecha) |>
  summarise(across(everything(),
    list(
      media = ~mean(., na.rm = TRUE),
      sd = ~sd(., na.rm = TRUE),
      min = ~min(., na.rm = TRUE),
      max = ~max(., na.rm = TRUE),
      n = ~n()
    )
  ),
```

```

    .names = "{.col}_{.fn}"
  ))

print(stats_por_activo)

```

Tiempo: ~10 minutos.

Paso 3: Visualizaciones

```

portafolio_retornos |>
  select(-fecha) |>
  pivot_longer(everything(), names_to = "activo", values_to = "retorno") |>
  ggplot(aes(x = retorno, fill = activo)) +
  geom_histogram(alpha = 0.45, binwidth = 0.01) +
  facet_wrap(~activo, scales = "free") +
  theme_minimal() +
  labs(title = "Distribución de Retornos Mensuales (2012-2021)",
       x = "Retorno", y = "Frecuencia")

portafolio_retornos |>
  mutate(fecha = as.Date(fecha)) |>
  pivot_longer(-fecha, names_to = "activo", values_to = "retorno") |>
  ggplot(aes(x = fecha, y = retorno, color = activo)) +
  geom_line(alpha = 0.7) +
  facet_wrap(~activo, scales = "free_y") +
  theme_minimal() +
  labs(title = "Retornos Mensuales en el Tiempo",
       x = "Fecha", y = "Retorno")

```

Tiempo: ~12 minutos.

Paso 4: Matriz de correlación y covarianza

```

retornos_matrix <- portafolio_retornos |>
  select(-fecha) |>
  as.matrix()

correlacion <- cor(retornos_matrix)
corrplot(correlacion, method = "circle", type = "upper",
         title = "Matriz de Correlaciones entre Activos",
         mar = c(0, 0, 2, 0))

covarianza <- cov(retornos_matrix)
print("Matriz de Covarianza:")
print(round(covarianza, 6))

correlacion_df <- as.data.frame(correlacion) |>

```

```
rownames_to_column("activo1") |>
pivot_longer(-activo1, names_to = "activo2", values_to = "corr") |>
filter(activo1 < activo2) |>
arrange(desc(abs(corr)))

kable(correlacion_df, digits = 3, caption = "Correlaciones entre Activos")
```

Tiempo: ~15 minutos.

Total Parte 1 Manual: ~40 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Necesito un EDA completo de un portafolio de 5 activos desde portafolio_retornos.csv:
 1. Estadística descriptiva (media, sd, quantiles, valores faltantes) 2. Histogramas de retornos por activo 3. Series temporales de retornos 4. Matriz de correlación y covarianza con visualización”

Resultado (minutos):

Claude Code genera un script optimizado con tidyverse + corrplot + ggplot2.

Tiempo: ~5 minutos (lectura + ajustes).

Comparación: Manual vs IA (Parte 1)

Aspecto	Manual	Con IA
Tiempo	~40 min	~5 min
Cobertura	Buena, incompleta	Exhaustiva
Debugging	Manual, prueba-error	IA propone soluciones
Calidad código	Iterativo	Producción

Parte 2 — Markowitz: Optimización de la Frontera Eficiente

¿Por qué Markowitz primero?

Markowitz es la **solución óptima exacta** (analítica). Entender cómo se resuelve es fundamental antes de aproximarla con simulación.

Enfoque Manual

Paso 1: Preparar datos y matriz de covarianza

```
portafolio_retornos <- read_csv("data/portafolio_retornos.csv")

retornos_matrix <- portafolio_retornos |>
  select(-fecha) |>
  as.matrix()

retornos_esperados <- colMeans(retornos_matrix)
cov_matrix <- cov(retornos_matrix)
tasa_libre_riesgo <- 0.02 / 12
```

Tiempo: ~5 minutos.

Paso 2: Formular el problema de Markowitz

La frontera eficiente resuelve:

$$\min_w [w^T \Sigma w]$$

sujeto a: - $w^T r = r_p$ (retorno objetivo) - $\sum w_i = 1$ (pesos suman 1) - $w_i \geq 0$ (long-only, sin venta corta)

Paso 3: Instalar quadprog y resolver

```
# install.packages("quadprog")
library(quadprog)
```

```

n_activos <- ncol(retornos_matrix)
retornos_objetivo <- seq(0.08, 0.20, length.out = 50) / 12 # mensuales

frontera_markowitz <- data.frame(
  retorno = numeric(length(retornos_objetivo)),
  volatilidad = numeric(length(retornos_objetivo)),
  pesos = matrix(NA, nrow = length(retornos_objetivo), ncol = n_activos)
)

for (i in seq_along(retornos_objetivo)) {
  r_obj <- retornos_objetivo[i]

  # Formulación quadprog
  H <- 2 * cov_matrix
  f <- rep(0, n_activos)

  # Restricciones: sum(w)=1, w'*r=r_obj, w>=0
  A <- cbind(
    rep(1, n_activos),      # suma = 1
    retornos_esperados,     # retorno objetivo
    diag(n_activos)        # w >= 0
  )

  b0 <- c(1, r_obj, rep(0, n_activos))

  resultado <- solve.QP(H, f, t(A), b0, meq = 2)

  w_opt <- resultado$solution
  vol_opt <- sqrt(t(w_opt) %*% cov_matrix %*% w_opt) * sqrt(12)

  frontera_markowitz$retorno[i] <- r_obj * 12
  frontera_markowitz$volatilidad[i] <- vol_opt
  frontera_markowitz$pesos[i, ] <- w_opt
}

# Identificar portafolio tangente (máx Sharpe)
frontera_markowitz$sharpe <- (frontera_markowitz$retorno - 0.02) / frontera_markowitz$volatilidad
idx_tangente <- which.max(frontera_markowitz$sharpe)
portafolio_tangente <- frontera_markowitz[idx_tangente, ]

```

Tiempo: ~20 minutos (comprensión de quadprog, debugging de restricciones).

Paso 4: Visualizar la frontera exacta

```

frontera_markowitz |>
  ggplot(aes(x = volatilidad, y = retorno)) +
  geom_line(size = 1.2, color = "darkred") +

```

```
geom_point(data = portafolio_tangente, color = "red", size = 4) +  
theme_minimal() +  
labs(title = "Frontera Eficiente de Markowitz (Exacta)",  
      x = "Volatilidad Anual (%)",  
      y = "Retorno Anual Esperado (%)")
```

Tiempo: ~5 minutos.

Total Parte 2 Manual: ~30 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Calcula la frontera eficiente de Markowitz usando quadprog: - Long-only (pesos entre 0 y 1, suman 1) - Barre 50 puntos de retorno objetivo entre 8 % y 20 % anual - Para cada punto, minimiza varianza sujeto al retorno objetivo - Identifica el portafolio tangente (máximo Sharpe ratio) - Grafica la frontera con el punto tangente marcado en rojo”

Resultado (minutos):

Claude Code maneja la sintaxis tediosa de `quadprog` automáticamente.

Tiempo: ~5 minutos (lectura + ajustes).

Parte 3 — Simulación: Aproximar la Frontera por Monte Carlo

¿Por qué simulación después de Markowitz?

Ahora que conoces la solución *exacta*, usamos simulación para **validar**: “¿qué tan buena es una aproximación por fuerza bruta?”

Enfoque Manual

Paso 1: Simular miles de portafolios aleatorios

```
set.seed(42)

n_sims <- 10000
n_activos <- ncol(retornos_matrix)

resultados_sim <- data.frame(
  retorno_anual = numeric(n_sims),
  volatilidad_anual = numeric(n_sims),
  sharpe_ratio = numeric(n_sims)
)

pesos_matrix <- matrix(NA, nrow = n_sims, ncol = n_activos)

for (i in 1:n_sims) {
  # Generar pesos aleatorios long-only
  w <- runif(n_activos)
  w <- w / sum(w)

  # Retorno esperado del portafolio
  ret_port <- sum(w * retornos_esperados) * 12

  # Volatilidad del portafolio
  vol_port <- sqrt(t(w) %*% cov_matrix %*% w) * sqrt(12)

  # Sharpe ratio
```

```

sharpe <- (ret_port - 0.02) / vol_port

resultados_sim$retorno_anual[i] <- ret_port
resultados_sim$volatilidad_anual[i] <- vol_port
resultados_sim$sharpe_ratio[i] <- sharpe
pesos_matrix[i, ] <- w
}

# Mejor portafolio simulado
idx_mejor <- which.max(resultados_sim$sharpe_ratio)
mejor_sim <- resultados_sim[idx_mejor, ]
mejor_pesos_sim <- pesos_matrix[idx_mejor, ]

```

Tiempo: ~15 minutos.

Paso 2: Visualizar la nube

```

resultados_sim |>
  ggplot(aes(x = volatilidad_anual, y = retorno_anual, color = sharpe_ratio)) +
  geom_point(alpha = 0.5, size = 1) +
  scale_color_viridis_c(direction = 1) +
  geom_point(data = mejor_sim, color = "red", size = 4, shape = 21, stroke = 2) +
  theme_minimal() +
  labs(title = "Frontera Eficiente Simulada (10,000 portafolios)",
        x = "Volatilidad Anual (%)",
        y = "Retorno Anual Esperado (%)",
        color = "Sharpe Ratio")

```

Tiempo: ~10 minutos.

Total Parte 3 Manual: ~25 minutos.

Enfoque Asistido con Claude Code

Prompt:

“Simula 10,000 portafolios long-only con pesos aleatorios. Para cada uno: retorno esperado (anualizado), volatilidad (anualizada), Sharpe ratio. Identifica el mejor portafolio por Sharpe. Grafica la nube coloreada por Sharpe, marcando el mejor con un punto rojo.”

Resultado (minutos):

Claude genera el loop o versión vectorizada automáticamente.

Tiempo: ~3 minutos.

Parte 4 — Comparación: ¿Cuán Buena es la Simulación?

El Gráfico Overlay: Validación Visual

Ahora superpones ambos resultados:

```
# Gráfico combinado: nube simulada + frontera exacta
ggplot() +
  geom_point(data = resultados_sim,
             aes(x = volatilidad_anual, y = retorno_anual, color = sharpe_ratio),
             alpha = 0.4, size = 1) +
  geom_line(data = frontera_markowitz,
            aes(x = volatilidad, y = retorno),
            color = "red", size = 1.5, linetype = "dashed") +
  geom_point(data = mejor_sim,
             color = "darkred", size = 4, shape = 21, stroke = 2) +
  geom_point(data = portafolio_tangente,
             color = "darkgreen", size = 4, shape = 23, stroke = 2) +
  scale_color_viridis_c(direction = 1) +
  theme_minimal() +
  labs(title = "Frontera Eficiente: Simulación (azul) vs Markowitz (roja)",
       x = "Volatilidad Anual (%)",
       y = "Retorno Anual Esperado (%)",
       color = "Sharpe (Sim)")
```

Análisis: ¿Qué Observas?

1. La nube simulada toca la frontera roja? → Depende de N (10k no es suficiente)
2. El mejor punto simulado tangente de Markowitz? → Normal, es una aproximación
3. ¿Cuán lejano está el mejor simulado del tangente? → Mide la “brecha de optimización”

Cálculo de la brecha:

```
# Distancia euclidiana entre mejor simulado y tangente de Markowitz
brecha_retorno <- abs(mejor_sim$retorno_anual - portafolio_tangente$retorno[idx_tangente])
brecha_vol <- abs(mejor_sim$volatilidad_anual - portafolio_tangente$volatilidad[idx_tangente])
```

```
brecha_sharpe <- abs(mejor_sim$sharpe_ratio - portafolio_tangente$sharpe[idx_tangente])

cat("Brecha de optimización:\n")
cat("Retorno:", brecha_retorno, "\n")
cat("Volatilidad:", brecha_vol, "\n")
cat("Sharpe:", brecha_sharpe, "\n")
cat("\nConclusión: Con 10k simulaciones, el error es de ~",
    round(brecha_sharpe / portafolio_tangente$sharpe[idx_tangente] * 100, 2), "%\n")
```

Enfoque con Claude Code

```
# Prompt: "Superpón nube simulada + frontera Markowitz.
# Marca mejor simulado en rojo, tangente en verde.
# Calcula distancia euclidiana entre ambos puntos."
```

Tiempo: ~10 minutos total.

Parte 5 — Servicing: Publicar tu Proyecto en GitHub Pages

¿Por qué servir importa?

Hasta aquí tienes un análisis que funciona en tu máquina. **Servicing** = convertir ese análisis en una **página web interactiva** que otros puedan usar sin ejecutar código.

La buena noticia: no necesitas servidor costoso ni configuración compleja. GitHub Pages + Quarto = gratis + automático.

El flujo completo (Crear Proyecto → GitHub Pages)

Paso 1: Crear tu repositorio local

En tu terminal, crea una carpeta para tu proyecto:

```
# Crea la carpeta
mkdir portafolio-optimo-r
cd portafolio-optimo-r

# Inicializa git
git init

# Descarga el CSV de datos desde la clase y colócalo en una subcarpeta
mkdir data
# (Copia portafolio_retornos.csv a data/)
```

Paso 2: Crear la estructura de Quarto

Crea un archivo `_quarto.yml` en la raíz del proyecto (personaliza con tu nombre):

```
project:
  type: website
  output-dir: docs

website:
  title: "Análisis de Portafolios Óptimos"
```

```

navbar:
  left:
    - text: "Inicio"
      href: index.qmd
    - text: "Autor"
      href: about.qmd

format:
  html:
    theme: cosmo
    toc: true
    toc-depth: 2

```

Paso 3: Crear tu archivo principal de análisis

Creas `index.qmd` con tus 4 partes (EDA, Markowitz, Simulación, Comparación):

```

---
title: "Construcción de Portafolios Óptimos"
subtitle: "EDA → Markowitz → Simulación → Validación"
author: "[Tu Nombre]"
date: last-modified
format:
  html:
    toc: true
    code-fold: false
---

# Introducción

[Tu contexto del problema]

# Parte 1 - EDA

[Tu código de EDA aquí]

# Parte 2 - Markowitz

[Tu código de Markowitz aquí]

# Parte 3 - Simulación

[Tu código de simulación aquí]

# Parte 4 - Comparación

[Tus gráficos de validación aquí]

```

Paso 4: Agregar la Parte 5 — Interactividad

En el mismo archivo `index.qmd`, agrega un bloque `{ojs}` (Observable JavaScript) con sliders interactivos:

```
// 1. Cargar datos
portafolio_data = FileAttachment("data/portafolio_retornos.csv").csv()

// 2. Crear controles interactivos (sliders para cada activo)
peso_spy = Inputs.range([0, 100], {value: 20, step: 1, label: "SPY (%)"})
peso_efa = Inputs.range([0, 100], {value: 25, step: 1, label: "EFA (%)"})
peso_ijs = Inputs.range([0, 100], {value: 15, step: 1, label: "IJS (%)"})
peso_eem = Inputs.range([0, 100], {value: 20, step: 1, label: "EEM (%)"})
peso_agg = Inputs.range([0, 100], {value: 20, step: 1, label: "AGG (%)"})

// 3. Normalizar pesos (asegurar que sumen 100%)
pesos_raw = [peso_spy, peso_efa, peso_ijs, peso_eem, peso_agg]
suma_pesos = pesos_raw.reduce((a, b) => a + b, 0)
pesos = pesos_raw.map(w => w / suma_pesos * 100)

// 4. Calcular retorno/volatilidad esperados (precomputa de tu análisis)
retornos_medios = [0.12, 0.09, 0.14, 0.15, 0.04] // anuales
sharpe_resultado = (pesos[0] * retornos_medios[0] + ...) / volatilidad_portfolio

// 5. Mostrar resultados
html`
<div style="padding: 20px; background: #f0f8ff; border-radius: 8px;">
  <h3>Tu Portafolio Personalizado</h3>
  <p><strong>Retorno Esperado:</strong> ${retorno_total.toFixed(2)}% anual</p>
  <p><strong>Volatilidad:</strong> ${volatilidad_portfolio.toFixed(2)}% anual</p>
  <p><strong>Sharpe Ratio:</strong> ${sharpe_resultado.toFixed(2)}</p>
</div>
`
```

Paso 5: Renderizar localmente

En tu terminal:

```
# Renderiza el proyecto a HTML (se guarda en docs/)
quarto render

# Opcionalmente, previsualiza localmente
quarto preview
```

Abre `docs/index.html` en tu navegador para verificar que todo funciona.

Paso 6: Crear repositorio en GitHub

1. Ve a github.com y crea un nuevo repositorio público
2. Nómbralo (ej: `portafolio-optimo-r`)

3. **NO inicialices con README** (ya tienes archivos locales)

Paso 7: Vincular repositorio local con GitHub

En tu terminal:

```
# Agrega el repositorio remoto
git remote add origin https://github.com/[TU_USUARIO]/portafolio-optimo-r.git

# Agrega todos los archivos
git add .

# Commit inicial
git commit -m "feat: Initial portfolio analysis project"

# Push a main branch
git branch -M main
git push -u origin main
```

Paso 8: Habilitar GitHub Pages

En GitHub:

1. Ve a **Settings** → **Pages**
2. En “Build and deployment”, elige:
 - Source: **Deploy from a branch**
 - Branch: **main** / folder: **/docs**
3. Haz clic en **Save**

¡Listo! Tu sitio estará disponible en: [https://\[TU_USUARIO\].github.io/portafolio-optimo-r/](https://[TU_USUARIO].github.io/portafolio-optimo-r/)

Flujo de actualización futuro

Después de esto, cada vez que quieras actualizar:

```
# 1. Modifica tus archivos .qmd
# 2. Renderiza
quarto render

# 3. Commit + push
git add .
git commit -m "update: Refina simulación con 50k portafolios"
git push origin main

# GitHub Pages se actualiza automáticamente en ~30 segundos
```

¿Cómo construir la página Observable JS?

Estructura recomendada:

1. **Carga de datos** (FileAttachment)
2. **Controles interactivos** (Inputs.range, Inputs.select)
3. **Lógica reactiva** (cálculos que se actualizan al cambiar sliders)
4. **Visualización** (HTML o gráficos con Observable Plot/D3)

Ejemplo simplificado:

```
// Cargar datos
datos = FileAttachment("data/portafolio_retornos.csv").csv()

// Slider reactivo
mi_peso = Inputs.range([0, 100], {label: "Mi peso en SPY"})

// Cálculo que reacciona al slider
retorno_calculado = mi_peso / 100 * retorno_spy

// Mostrar resultado
html`<p>Retorno: ${retorno_calculado.toFixed(2)}%</p>`
```

Recursos adicionales:

- [Observable Help](#)
- [Quarto + Observable JS](#)
- [Observable Plot \(gráficos\)](#)
- [D3.js \(avanzado\)](#)

Comparación: Enfoque manual vs con IA

Tarea	Manual	Con IA
Estructurar proyecto Quarto	20 min	3 min (1 prompt)
Escribir Observable JS	40 min	5 min (pedir código)
Configurar GitHub Pages	15 min	5 min (guía)
Renderizar + publicar	5 min	2 min
Total	~80 min	~15 min

Checklist antes de entregar

- ☐ Carpeta local con estructura: `_quarto.yml`, `index.qmd`, `data/portafolio_retornos.csv`
- ☐ `quarto render` genera `docs/index.html` sin errores

- ☐ Todas las 4 partes (EDA, Markowitz, Simulación, Comparación) están en `index.qmd`
 - ☐ Bloque `{ojs}` con sliders funciona en el navegador local
 - ☐ Repositorio GitHub creado y vinculado
 - ☐ GitHub Pages habilitado en Settings → Pages
 - ☐ Página viva en `https://[usuario].github.io/[repo-name]/`
 - ☐ Sliders interactivos funcionan en la página pública
 - ☐ URL compatible en tu entrega final
-

Ejemplo de estructura final

```
portafolio-optimo-r/  
  _quarto.yml          # Configuración de Quarto  
  index.qmd            # Tu análisis completo (Partes 1-5)  
  about.qmd            # (Opcional) Página "Acerca de"  
  data/  
    portafolio_retornos.csv # Tu CSV de datos  
  docs/  
    index.html          # Generado por `quarto render`  
    ...  
  .gitignore           # (Opcional) para ignorar /docs en git
```

Próximo paso

1. Completa Partes 1-4 en tu `.qmd` local
 2. Agrega el bloque `{ojs}` interactivo
 3. Prueba con `quarto preview`
 4. Sigue los pasos 6-8 para GitHub + Pages
 5. Comparte tu URL en la entrega final
-

Tabla de Ventajas: Manual vs IA

Tiempos estimados para completar la actividad completa (incluyendo crear tu propio proyecto GitHub Pages):

Aspecto	Manual (Total)	Con IA (Total)
Parte 1 (EDA)	40 min	5 min
Parte 2 (Markowitz)	30 min	5 min
Parte 3 (Simulación)	25 min	3 min
Parte 4 (Comparación)	15 min	8 min
Parte 5 (Proyecto + GitHub Pages)	40 min	15 min
TOTAL	~150 min	~36 min

Nota: Parte 5 ahora incluye crear estructura Quarto, escribir Observable JS, y publicar en GitHub Pages (antes no estaba). El enfoque con IA sigue siendo **4x más rápido**.

Actividad para Estudiantes

Instrucciones Generales

Elige UNA de las 3 opciones. Todas conducen al mismo resultado: análisis completo + página publicada.

Opción A: Maximalista (Aprender escribiendo código)

Sigue todas las 5 partes escribiendo código tú mismo. Usa Claude Code solo para debugging o sugerencias puntuales.

Entregas: script R/Quarto comentado, gráficos (nube + frontera), página interactiva.

Opción B: Híbrida (Equilibrada)

1. Parte 1: Escribes EDA (10 min)
2. Parte 2: Pides a Claude que genere Markowitz (5 min lectura)
3. Parte 3: Escribes simulación con ayuda (15 min)
4. Parte 4: Análisis de brecha con Claude (10 min)
5. Parte 5: Claude genera Observable JS (5 min ajustes)

Opción C: Minimalista (Prompt-Driven)

Pídele a Claude Code el flujo completo en 1-2 prompts. Claude genera ~800 líneas en ~5 minutos. Tu rol: entender, interpretar, reflexionar.

Rúbrica de Evaluación

Criterio	Excelente	Bueno	Aceptable
EDA	Stats + 3+ gráficos + matriz de correlación clara	2 gráficos + correlación	Solo summary()
Markowitz	Frontera exacta (50+ puntos), long-only correcto, portafolio tangente identificado	20-50 puntos, correcto	Incompleto/erróneo

Criterio	Excelente	Bueno	Aceptable
Simulación	5k+ portafolios, pesos long-only correctos, nube visualizada	1-5k portafolios, correcto	<1k o errores
Comparación	Overlay claro, brecha calculada, interpretación de diferencias	Overlay presente	Ausente
Servicing	Sliders funcionales, calcula dinámicamente, responsive	Sliders básicos	Ausente
Interpretación	Explica por qué Markowitz > simulación, cuándo usar cada uno	Menciona diferencias	Mínima
Uso de IA	Estratégico (boilerplate, debugging, iteración)	Parcial	Mínimo

Entrega

Tu entrega debe incluir:

1. **URL de GitHub** del repositorio con tu proyecto (público)
 - Carpeta con estructura Quarto correcta
 - Archivo `index.qmd` con todas las 5 partes
 - Carpeta `data/` con tu CSV
2. **URL de GitHub Pages** de tu proyecto en vivo
 - Demuestra que la página está publicada
 - Verifica que los sliders funcionan
3. **Reflexión escrita** (1 página, formato libre):
 - ¿Qué estrategia elegiste (A: Maximalista, B: Híbrida, C: Minimalista)?
 - ¿Cómo usaste IA en cada parte? (Sé específico con prompts si es relevante)
 - ¿Cuál fue la brecha de optimización que encontraste? (Diferencia Markowitz vs simulación)
 - ¿Qué aprendiste sobre GitHub Pages y Quarto?
 - ¿Cuáles fueron los principales desafíos técnicos?
4. **Archivo de código comentado** (opcional pero recomendado):
 - Exporta tu `index.qmd` desde GitHub
 - Incluye comentarios que expliquen decisiones clave

Troubleshooting Común

“Error: quarto render falla”

Posible causa: Faltan librerías de R.

```
# Instala las que faltan
Rscript -e "install.packages('tidyverse'); install.packages('quadprog')"
```

“GitHub Pages muestra 404”

Pasos a verificar:

1. En tu repo, ve a **Settings** → **Pages**
2. Confirma que Source es “Deploy from a branch” + “main” + “/docs”
3. Espera 1-2 minutos (GitHub tarda en actualizar)
4. La URL debe ser: `https://[usuario].github.io/[repo-name]/`

“Los sliders Observable no cargan datos”

Causa común: El archivo CSV no está en la ruta correcta.

```
// Incorrecto
datos = FileAttachment("portafolio_retornos.csv").csv()

// Correcto (asume que quarto render está en la raíz)
datos = FileAttachment("data/portafolio_retornos.csv").csv()
```

“La página renderiza pero los cálculos no funcionan”

Verifica:

1. ¿El CSV se carga? Abre browser console (F12) y busca errores
2. ¿Los valores son NaN? Revisa que los nombres de columnas coincidan:

```
names(portafolio_retornos) # En R
```

3. ¿Falta variable en Observable? Usa `console.log(variable)` en {ojs}

“¿Cómo actualizar la página después de cambios?”

```
# 1. Modifica tu .qmd
# 2. Renderiza
quarto render

# 3. Commit + push
git add .
git commit -m "update: [descripción]"
git push origin main

# GitHub Pages se actualiza en ~30 segundos
# Recarga la página en tu navegador (Ctrl+Shift+R para forzar caché)
```

Reflexión Final: IA como Herramienta, No Reemplazo

Usa IA para:

- Boilerplate (cargas, transformaciones estándar)
- Exploración de sintaxis (“¿cómo uso quadprog?”)
- Debugging (“¿por qué falla esta restricción?”)
- Observable JS (sintaxis compleja)
- Documentación y comentarios

NO uses IA para:

- Decisiones de estrategia (tú eliges qué validar)
 - Validación de resultados (verifica siempre que $\text{Sharpe} = (\text{ret} - \text{rf}) / \text{vol}$)
 - Comprensión (entiende el código antes de publicarlo)
 - Pensar (IA acelera ejecución, no estrategia)
-

Alternativas y Extensiones

Si quieres explorar más allá...

Opción 1: Agregar una API con plumber (R)

En lugar de solo Observable JS estático, puedes crear un backend interactivo:

```
# En R: instala.packages("plumber")
# Crea un archivo api.R con endpoints que reciban pesos
# y devuelvan retorno/volatilidad calculados

## @post /calcular-portafolio
function(pesos) {
  retorno <- sum(pesos * retornos_medios)
  return(list(retorno = retorno))
}
```

Luego desplégalo en [Shinyapps.io](https://shinyapps.io) o [Heroku](https://heroku.com) (gratuitamente).

Tiempo adicional: ~60 min de aprendizaje.

Opción 2: Usar Shiny en lugar de Observable JS

Si prefieres R puro, puedes usar `library(shiny)` en lugar de Observable JS:

```
library(shiny)

ui <- fluidPage(
  sliderInput("peso_spy", "SPY (%)", 0, 100, 20),
  textOutput("resultado")
)

server <- function(input, output) {
  output$resultado <- renderText({
    paste("Retorno:", input$peso_spy * 0.12)
  })
}

shinyApp(ui, server)
```

Deploy a shinyapps.io automáticamente.

Nota: Shiny tiene servidor gratuito limitado (25 horas/mes).

Opción 3: Versión en Python con Streamlit

Si prefieren Python en lugar de R:

```
import streamlit as st
import pandas as pd

peso_spy = st.slider("SPY (%)", 0, 100, 20)
st.write(f"Retorno estimado: {peso_spy * 0.12}%")
```

Deploy en [Streamlit Cloud](#) (gratis).

Tiempo adicional: ~45 min si ya conoces Python.

¿Cuál elegir?

Opción	Complejidad	Deploy	Coste	Recomendado para
Observable JS (Parte 5)	Baja	GitHub Pages	Gratis	Prototipos rápidos, estático
Shiny	Media	Shinyapps.io	Gratis (limitado)	Análisis interactivo, R-centric
Streamlit	Media	Streamlit Cloud	Gratis	Python-centric, máquinas ML
API + Backend	Alta	Heroku/Railway	Gratis (con límites)	Producción real

Recursos Adicionales

[Markowitz Modern Portfolio Theory](#)

[quadprog: Quadratic Programming](#)

[Claude Code Docs](#)

[Positron IDE](#)

[Observable JS + D3](#)